

Semantic Modeling & Schema Design



SoftUni Team
Technical Trainers



SoftUni



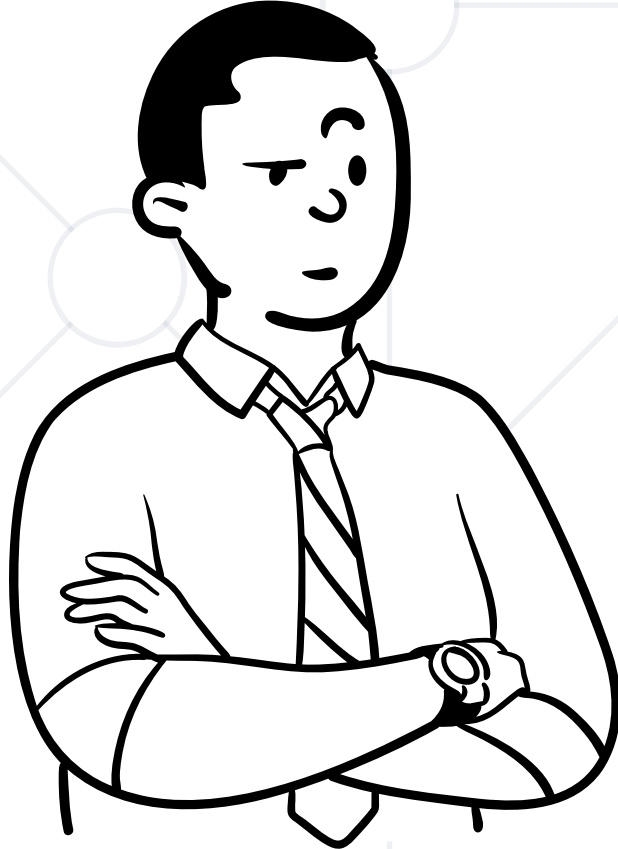
Software University

<https://about.softuni.bg>

1. The Excel Mindset vs. Database Mindset: Flat Tables vs. Relational Models.
2. The Star Schema: Facts and Dimensions.
3. Relationships: Keys, Cardinality (1:*), and Direction.
4. The Interface: Mastering the Model View.
5. Best Practices: The Waterfall Layout & Hiding Columns.



- Lecture 1: We set up the stack (Postgres -> PBI).
- Lecture 2: We cleaned data using SQL Views & Query Folding.



Now: We have clean tables.

How do we make them talk to each other?

The "Flat Table" Monster (The Problem)

- **Scenario:** In Excel, we are used to one giant table (VLOOKUP hell).
- **Example:** A table with 1 million rows containing: Date, Product Name, Product Color, Customer Name, City, Sales Amount.
- **Problem:** HUGE duplication. "Ivan" and "Sofia" are repeated 10,000 times..



Why not one big table?

- **Performance:** Text is heavy. Processing 1M repetitions of "Refrigerator" is slow.
- **Update Nightmare:** If "Ivan" changes his name, you must update 10,000 rows.
- **Confusion:** Hard to answer "How many unique customers do we have?".

✗ Flat Table (1M rows):

Date	Customer	City	Product	Color	Amount
2024-01-01	Ivan	Sofia	Laptop	Black	1200
2024-01-02	Ivan	Sofia	Mouse	White	25
2024-01-03	Ivan	Sofia	...	...	...

↓ "Ivan" and "Sofia" repeated 10,000 times

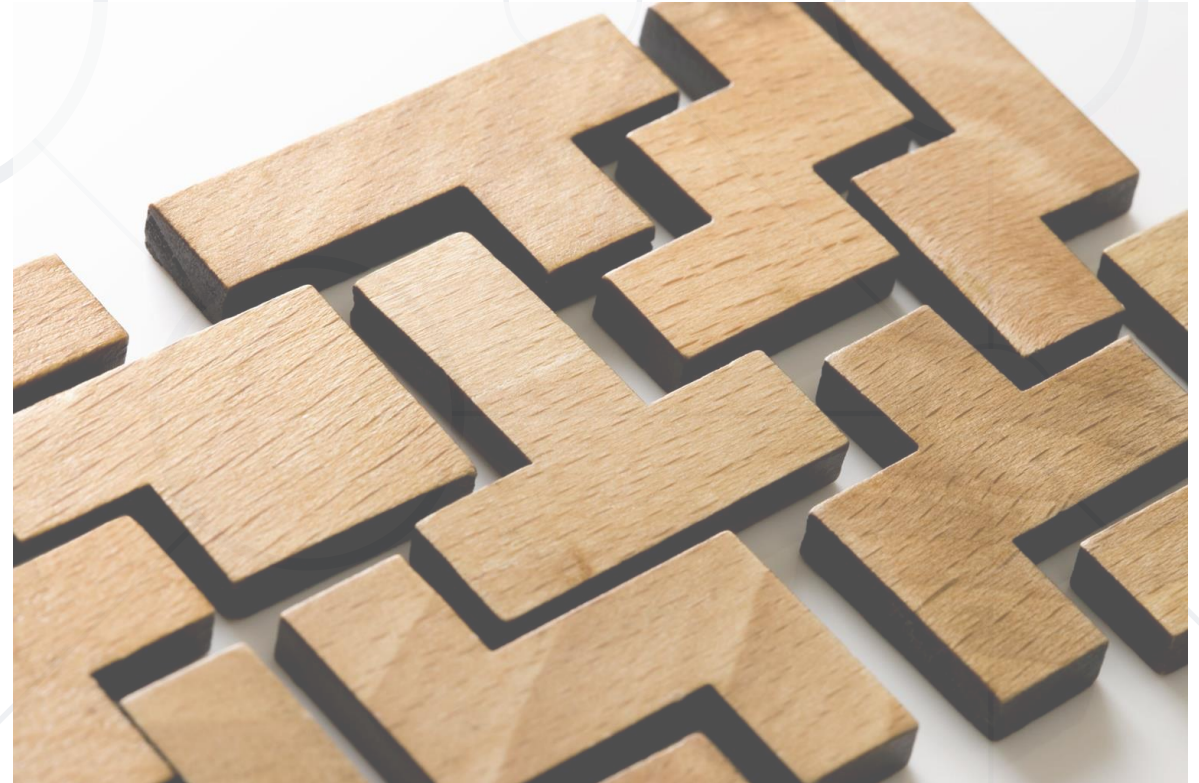
✓ Star Schema (3 tables):

Dim_Customer: Ivan → Sofia (stored ONCE)

Fact_Sales: Links to Customer_ID (just a number)

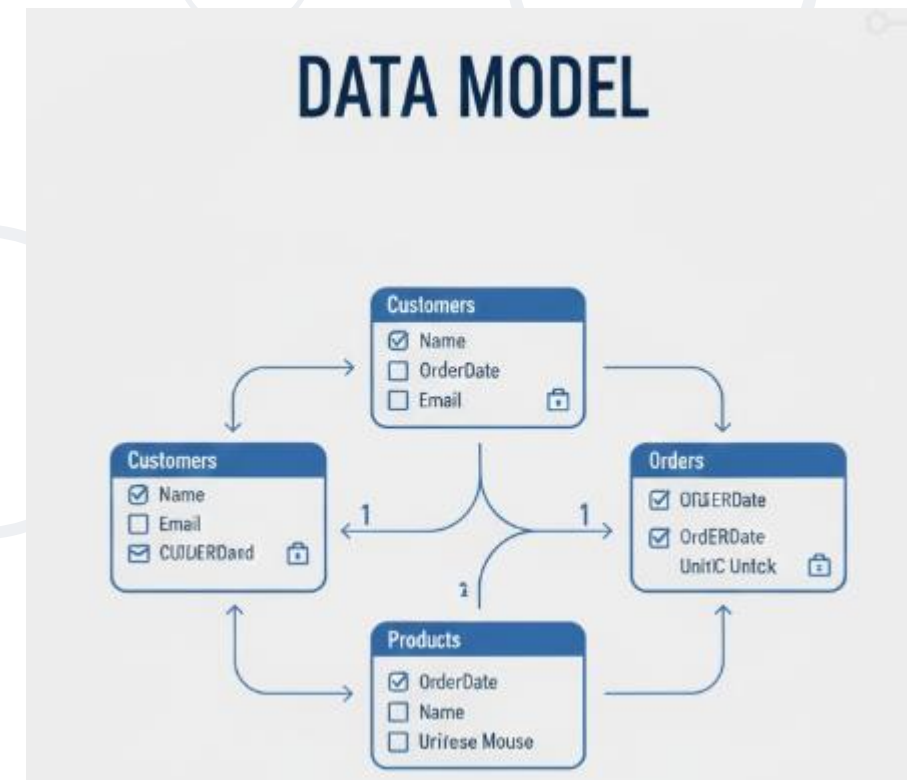
The Solution: Normalization (Modeling)

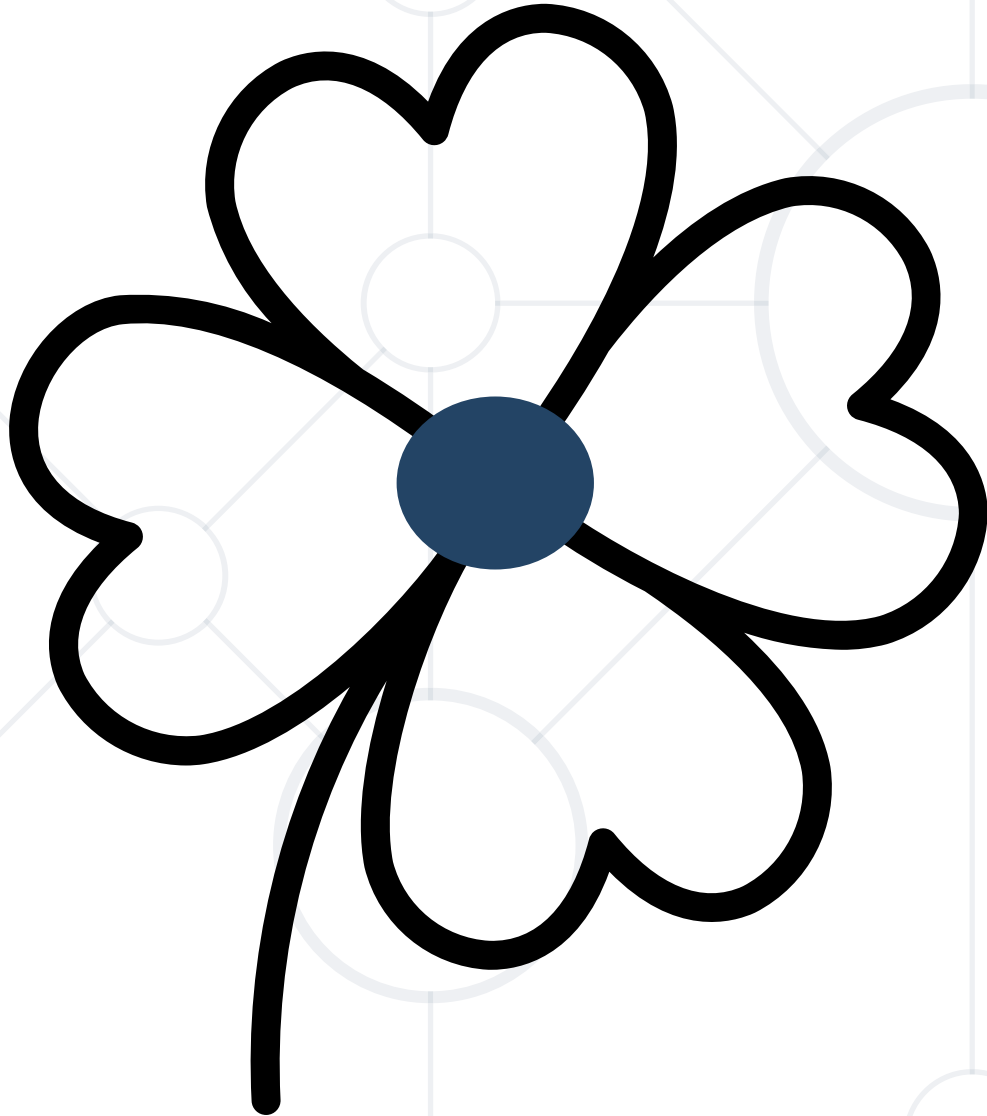
- **Concept:** Break the data into logical pieces.
- Keep "Who" separate from "What" and "When".



What is a Data Model?

- **Definition:** A collection of tables connected by relationships.
- It is the **Brain** of your report.
- If the Model is good, DAX is easy.
- If the Model is bad, DAX is impossible.





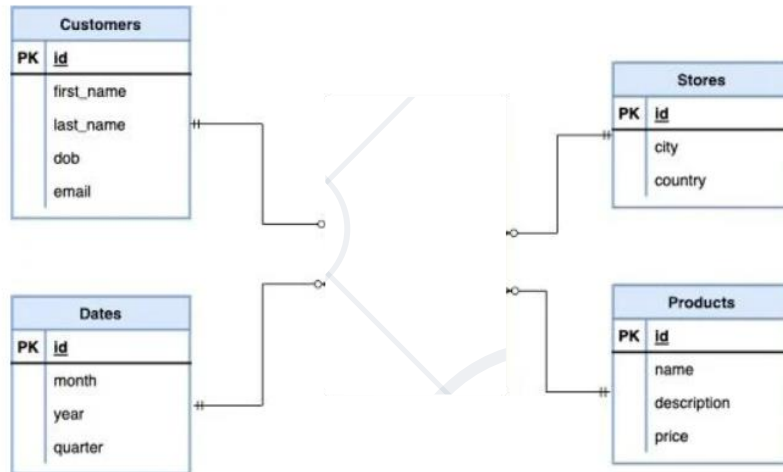
**We want to build a Star Schema.
The industry standard for BI.**

Why is it called a Star Schema?

- **Center:**
Fact Table.
- **Points:**
Dimension Tables.



Dimension Tables (The "Nouns")



Answers: Who? Where? What?

Content: Descriptive attributes (Text).

Examples: Customers, Products, Locations, Employees.

Characteristics: Short, Wide (many columns), Unique IDs.

Fact Tables (The "Verbs")

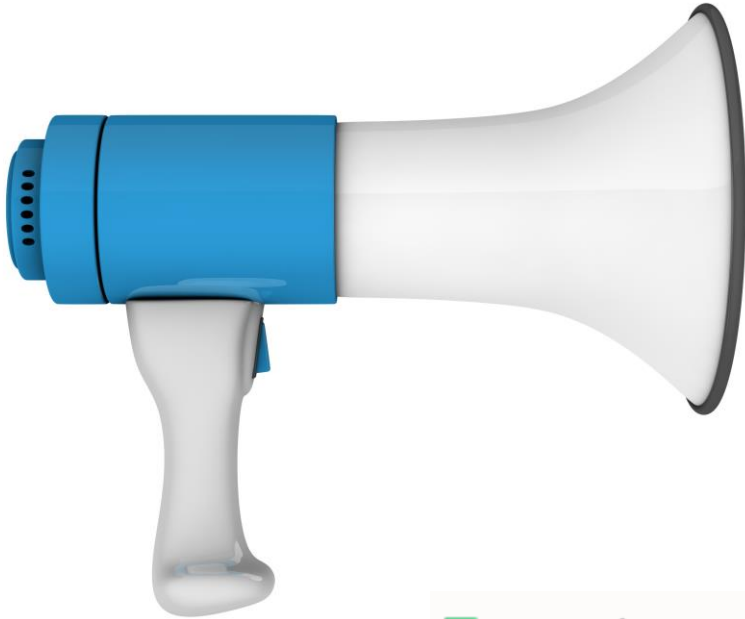
Orders	
PK	<u>id</u>
FK	customer_id
FK	product_id
FK	store_id
FK	date_id
	quantity
	amount

- **Answers:** How much? How many?
- **Content:** Numbers (Metrics) and Keys (IDs).
- **Examples:** Sales, Visits, Temperature Readings, Clicks.
- **Characteristics:** Long (millions of rows), Narrow (few columns).

Comparison Table

Aspect	Dimension	Fact
-----	-----	-----
Content	Text (Who/What)	Numbers (Metrics)
Size	Short, Wide	Long, Narrow
Example	Customers	Sales
Columns	Many (10-20)	Few (3-7)
Rows	Thousands	Millions
Can Sum?	 No	 Yes

How to distinguish them?



Test: Can you sum it up?

Price? Yes -> Fact.

Quantity? Yes -> Fact.

City Name? No (Sum of Sofia = Error) -> Dimension.

Date? Dimension (Special case).

- ✅ **FACT** if you can answer:
- Can I SUM it? (Revenue, Quantity)
 - Can I COUNT it? (Orders, Clicks)
 - Can I AVG it? (Temperature, Rating)

- ❌ **DIMENSION** if:
- It's descriptive text (Name, City)
 - It answers WHO/WHAT/WHERE/WHEN
 - Summing it makes no sense

Keys (The Glue)

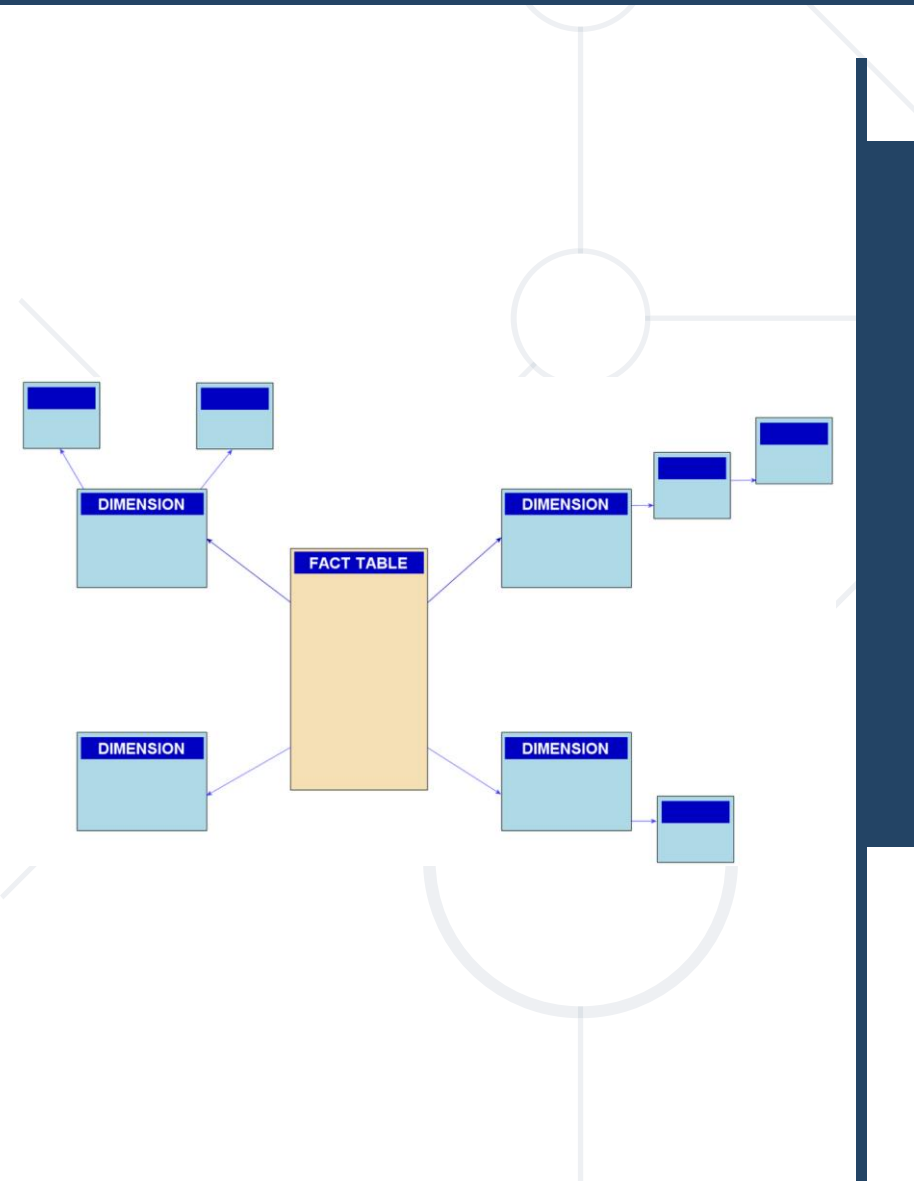
Primary Key (PK): Unique ID in the Dimension (e.g., `Employee_ID` in `Dim_Employee`).

Identifies one specific row.

Foreign Key (FK): The reference in the Fact Table (e.g., `Employee_ID` in `Fact_Sales`).

Can be repeated many times.

Snowflake Schema (The Variation)



- **Definition:** Dimensions connected to other Dimensions.
- Example: Product -> Category -> SubCategory.
- **Verdict:** Avoid it in Power BI if possible. It slows down performance.
- **Goal:** Collapse snowflakes into a single Star.

❌ Snowflake (Complicated):

Product → Category → SubCategory → Department
(3 hops to get from Product to Department)

✅ Star (Simple):

Product (includes Category, SubCategory, Department)
(1 table, direct access)

Live Demo



Definition: What does one row represent?

Example: One row = One receipt line? Or one row = Daily total?

Rule: You cannot mix granularities in the same Fact table.

Granularity = What does ONE row represent?

Example 1: Sales Fact

Granularity: One Transaction Line

OrderID	ProductID	Quantity	Amount
1001	P1	2	50

Example 2: Daily Sales Summary

Granularity: One Day

Date	Total_Sales
2024-01-01	5000

✗ NEVER MIX:

Date	Product	DailySales	TransactionID
------	---------	------------	---------------

(Can't tell if it's daily or transaction level!)

Rule: Every row = Same level of detail



Date Dimension (The Calendar)

- ❑ Every model needs a Calendar.
- ❑ Why? To analyze by Year, Quarter, Month, Fiscal Periods.
- ❑ Do not rely on the auto-date feature of Power BI (it's heavy).
- ❑ We will create a DAX Calendar later.

Why Calendar Table?

✗ Without: Can't group by Quarter, Fiscal Year, Week

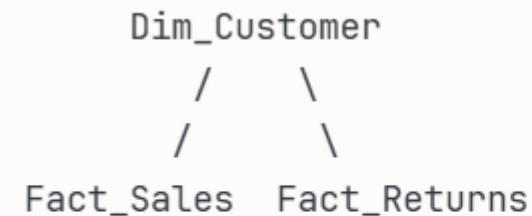
✓ With: Get these columns automatically:

- Year, Quarter, Month, Week
- Fiscal Year (if different from calendar)
- IsWeekend, IsHoliday
- Month Name (for sorting)



The Multi-Fact Challenge

- **Scenario:** Business asks: "Compare Sales vs. Returns by City."
- We have `Fact_Sales` and `Fact>Returns`.
- **Problem:** They are two separate processes.
- **Solution:** Do NOT connect Sales to Returns directly. Connect both to `Dim_Customers`.



- ✓ Both facts connect to same Customer dimension
- ✓ Can analyze: "Sales vs Returns by City"
- ✗ Never connect Sales → Returns directly!

- Drag and drop a line between two tables.
- Requirement: Both tables must share a common column (The Key).



Cardinality (The most important setting)

- Describes the numerical relationship between rows.
- Types: 1:*, *:1, 1:1, *:*.

One-to-Many (1:*)



- **The Gold Standard.**
- **Example:** One Customer has Many Orders.
- **Structure:** Unique ID in Dimension -> Multiple IDs in Fact.
- 99% of your relationships should be this.

Many-to-One (*:1)



Other side

- Same as **1:***, just looked at from the other side.
- Fact -> Dimension.

One-to-One (1:1)

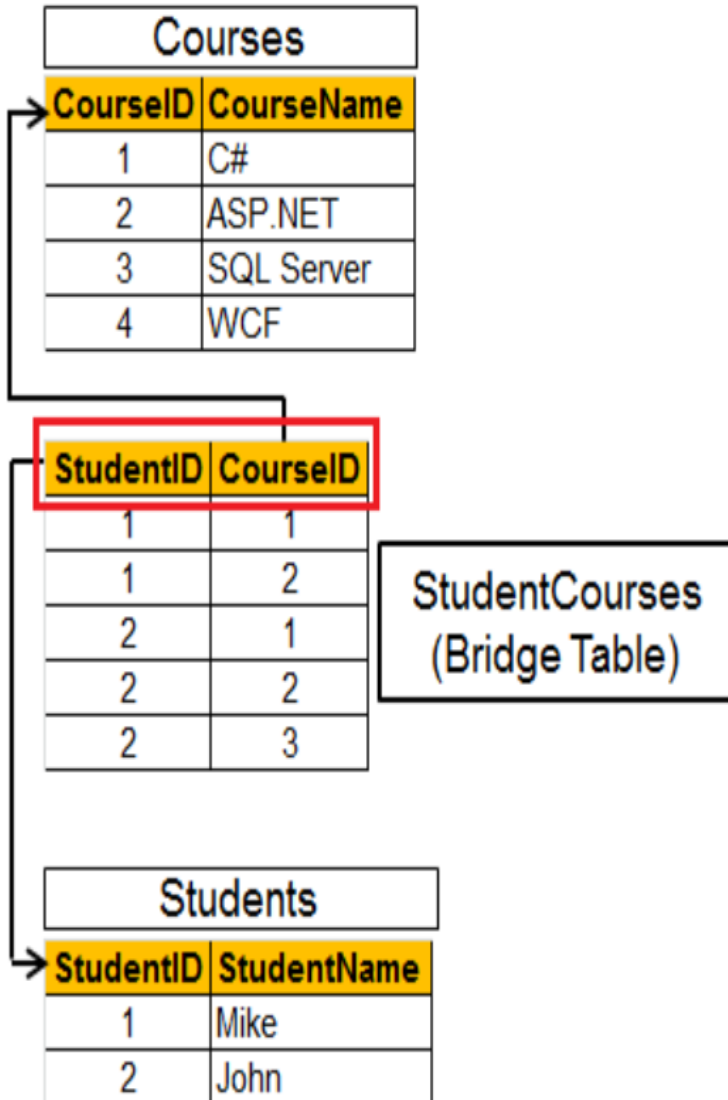


- Example: Employee -> Employee Details (Security badge info).
- Advice: If you have this, usually you should just merge them into one table.

Live Demo



Many-to-Many (:) - DANGER ZONE



- **Example:** Students and Classes (One student has many classes; One class has many students).
- **Power BI:** Supports it, but it creates ambiguity and strange results.
- **Avoid:** Unless you are an expert. Use a "Bridge Table" instead.

Many-to-Many Example:

Student ↔ Class

- One student has many classes
- One class has many students

✗ Direct M:M relationship = Ambiguous results

✓ Solution: Bridge Table

Student → Student_Class ← Class
(1:*) (Bridge) (*:1)

Bridge Tables: The M:M Solution

Problem Scenario:

Products sold through Multiple Channels- Laptop: Online, Retail, Wholesale- Mouse: Online, Retail

Bridge Table: Product_Channel

Product_ID	Channel_ID
Laptop	Online
Laptop	Retail
Laptop	Wholesale
Mouse	Online
Mouse	Retail

❌ Direct Many-to-Many (Avoid!):
Product ↔ Sales_Channel
(Ambiguous filtering, slow performance)

✅ Bridge Table Pattern:

Product —(1:*)→ Product_Channel ←—(*:1)— Sales_Channel
(Dimension) (Bridge) (Dimension)

Result: Two clean 1:* relationships = Clear filter flow! ✅



Single (-->): Filters flow from Dimension to Fact.

- Select "Sofia" -> Filters Sales.
- Select a Sale -> Does NOT filter Cities.

Both (<->): Filters flow both ways.

Why "Single" is better?

- **Performance:** Bi-directional is expensive.
- **Ambiguity:** Can create circular logic loops.



- **Rule:** Always use **Single** unless you have a very specific reason.

Active vs. Inactive Relationships

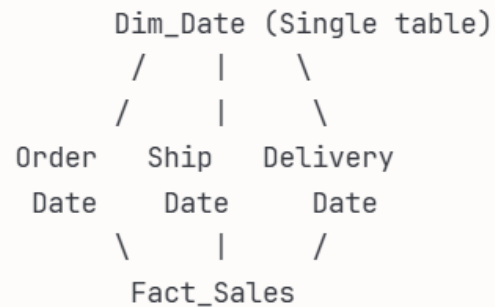


- You can have multiple relationships between two tables (e.g., Order Date vs. Ship Date).
- Only **ONE** can be Active (Solid line).
- Others are Inactive (Dotted line).
- Use **USERELATIONSHIP** in DAX to activate the dotted ones.

Role-Playing Dimensions: One Table, Many Roles

❌ Bad Solution: Create 3 separate Date tables
(Wastes space, inconsistent logic)

✅ Smart Solution: One Dim_Date, Multiple Roles



Relationships:

— Active: Order_Date (default analysis)

..... Inactive: Ship_Date

..... Inactive: Delivery_Date

How to Use Inactive Relationships?

Use USERELATIONSHIP() in DAX (we'll cover in Lecture 5)

Example:

Sales by Ship Date =

```
CALCULATE([Total Sales],
    USERELATIONSHIP(Fact_Sales[Ship_Date], Dim_Date[Date])
)
```

Real-World Problem: Fact_Sales has MULTIPLE dates:

- Order_Date (when customer ordered)
- Ship_Date (when warehouse shipped)
- Delivery_Date (when customer received)

Referential Integrity

- **Question:** What happens if we have a Sale for **Customer_ID = 999**, but **ID 999** does not exist in the Customer table?
- **Result:** Power BI shows a (Blank) row in reports.
- **Fix:** Ensure your SQL database cleans orphaned records!

Primary Table

CompanyId	CompanyName
1	Apple
2	Samsung

Related Table

CompanyId	ProductId	ProductName
1	1	iPhone
15	2	Mustang

Associated Record ✓

Orphaned Record ✗

Problem: Orphaned Records

Fact_Sales has Customer_ID = 999

But Dim_Customer doesn't have ID 999

Result in Power BI:

Customer	Sales	
-----	-----	
Ivan	1000	
Maria	2000	
(blank)	500	← Orphaned!

✓ Fix in SQL:

```
DELETE FROM fact_sales
WHERE customer_id NOT IN (SELECT id FROM dim_customer);
```



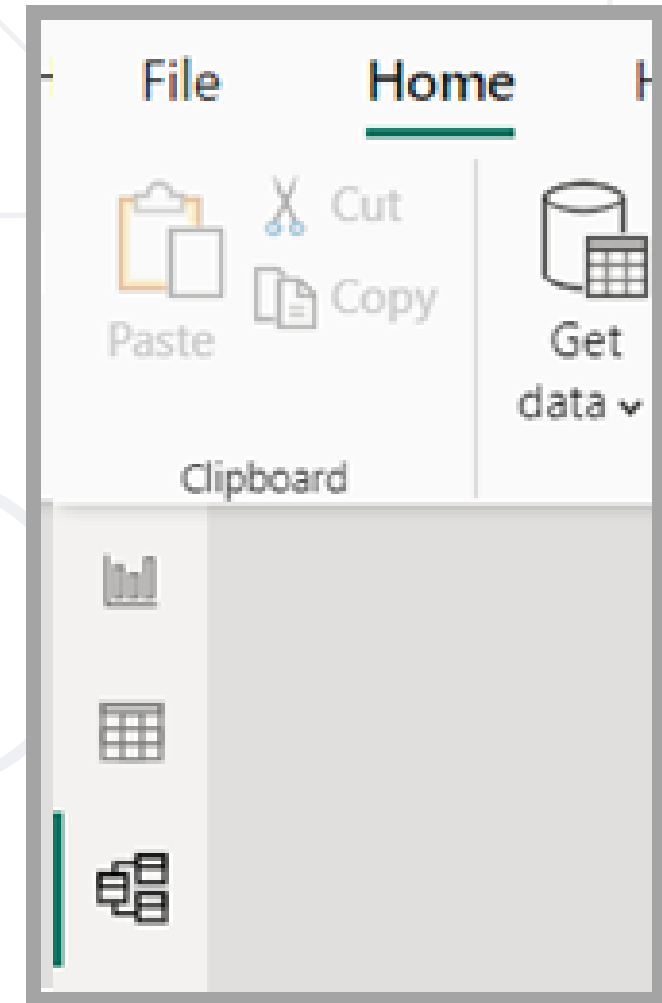
Live Demo



The Model View

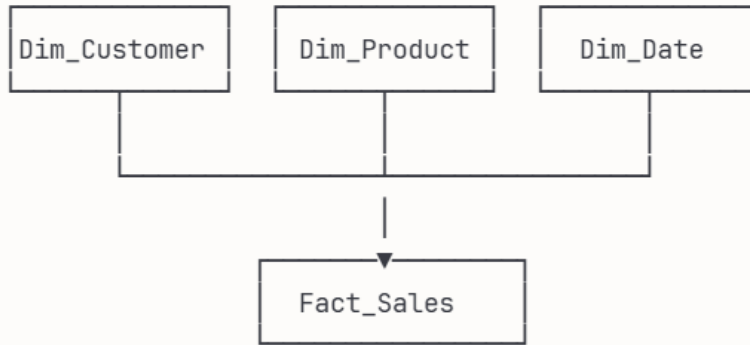
The third tab in the sidebar.

Features: Diagram view, Properties pane, Multi-select.



Layout Strategy: The "Waterfall"

Visual Layout:



"Filters flow DOWN like gravity" 🌊

- Don't leave tables scattered randomly.
- **Top Row:** Dimension Tables.
- **Bottom Row:** Fact Tables.
- **Why?** Visualizes how filters flow (Gravity flows down).

- **Description:** Add comments for other developers.
- **Synonyms:** For Q&A feature (AI).
- **Row Label:** Select which column represents the row.

Hiding Technical Columns



- **Rule:** End users should NEVER see ID columns (Keys).
- Right-click `employee_id` -> **Hide in Report View.**
- **Why?** Users will try to sum up IDs or get confused. Keep the UI clean.

Before Hiding Keys:

[Table shows: `customer_id`, `employee_id`, `order_id` visible]

✗ Users try to SUM `customer_id` = nonsense!

✗ Confusing numbers everywhere

After Hiding Keys:

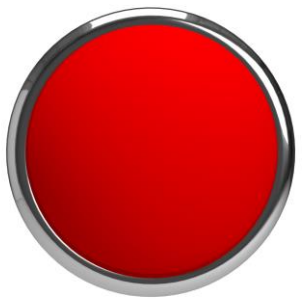
[Table shows: Customer Name, Employee Name, Order Date]

✓ Clean, user-friendly

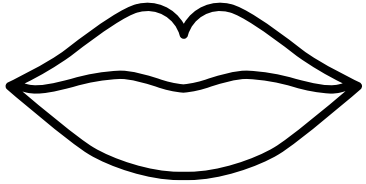
✓ Keys still work in relationships (hidden in backend)

Format Strings:

- Currency: Set to **BGN** or **\$**.
- Decimals: Limit to 0 or 2.
- Dates: Choose a short format (**dd.MM.yyyy**).



Do this in the Model View so it applies everywhere.



- Tell Power BI what the data represents.

- City Column: Set Category to "City" (Enables Map visuals).
- Image URL: Set Category to "Image URL" (Displays pictures).

Creating a Measure Table

- Don't store measures inside random tables.
- Create a dedicated table **_Measures**.
- How: Enter Data -> Name it **_Measures** -> Load -> Create measures there.

```
✓ Organized:
  _Measures
    └─ Total Sales
    └─ Total Profit
    └─ Avg Order Value

✗ Messy:
  Fact_Sales
    └─ OrderID
    └─ Total Sales (measure)
    └─ Amount
  Dim_Product
    └─ Avg Price (measure)
```

Benefit: All measures in one place = easier to find!

Live Demo



The Golden Rules of Modeling

- **Star Schema** > Snowflake > Flat Table.
- **1:*** Relationships are King.
- **Hide** keys from users.
- **Organize** layout (Waterfall).
- **Clean** data upstream (SQL Views) before modeling.

1. Relationships are King



2. Hide keys from users



3. Organize layout (Waterfall)



4. Clean data upstream (SQL Views) before modeling



- Leaving Many-to-Many relationships active.
- Using Bi-directional filtering everywhere "just to make it work".
- Leaving cryptic column names (`f_name`) visible.

Live Demo



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

